

ZYNova 2K26: C++ SOLUTIONS REFERENCE

This document provides optimized C++ implementations for the Zynova 2K26 Codathon challenge problems.

Level 0 - (Medium Complexity)

1. Cinema Dynamic Pricing

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int age, hour;
    if (!(cin >> age >> hour)) return 0;

    double price = 200.0;

    // Apply Age Discount
    if (age < 12) {
        price *= 0.5; // 50% discount
    } else if (age > 60) {
        price *= 0.7; // 30% discount
    }

    // Apply Time Surcharge
    if (hour >= 18 && hour <= 23) {
        price += 50.0;
    }

    cout << "Ticket Price: Rs. " << fixed << setprecision(0) << price << endl;

    return 0;
}
```

2. Smart Vending Machine

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int price, paid;
    if (!(cin >> price >> paid)) return 0;
```

```

int change = paid - price;
if (change < 0) {
    cout << "Insufficient Amount Paid" << endl;
    return 0;
}

int n10 = change / 10;
change %= 10;

int n5 = change / 5;
change %= 5;

int n2 = change / 2;
change %= 2;

int n1 = change;

cout << "Change: 10x" << n10 << ", 5x" << n5 << ", 2x" << n2 << ", 1x" << n1
<< endl;

return 0;
}

```

3. Robot Path Validator

```

#include <iostream>
#include <string>
#include <vector>
#include <set>

using namespace std;

int main() {
    int tx, ty;
    cin >> tx >> ty;

    int n;
    cin >> n;
    set<pair<int, int>> obstacles;
    for (int i = 0; i < n; ++i) {
        int ox, oy;
        cin >> ox >> oy;
        obstacles.insert({ox, oy});
    }

    string moves;
    cin >> moves;

    int cx = 0, cy = 0;

    // Check initial position (0,0) - though usually start is safe

```

```

if (cx == tx && cy == ty) {
    cout << "TARGET_REACHED" << endl;
    return 0;
}

for (char move : moves) {
    if (move == 'U') cy++;
    else if (move == 'D') cy--;
    else if (move == 'L') cx--;
    else if (move == 'R') cx++;

    if (cx < 0 || cx > 9 || cy < 0 || cy > 9) {
        cout << "OUT_OF_BOUNDS at (" << cx << "," << cy << ")" << endl;
        return 0;
    }

    if (obstacles.count({cx, cy})) {
        cout << "OBSTACLE at (" << cx << "," << cy << ")" << endl;
        return 0;
    }

    if (cx == tx && cy == ty) {
        cout << "TARGET_REACHED" << endl;
        return 0;
    }
}

cout << "INCOMPLETE at (" << cx << "," << cy << ")" << endl;

return 0;
}

```

4. Real-time Health Alert System

```

#include <iostream>
#include <vector>
#include <string>

using namespace std;

int main() {
    vector<int> hr(10);
    for (int i = 0; i < 10; ++i) cin >> hr[i];

    bool emergency = false;
    bool warning = false;

    for (int i = 0; i <= 7; ++i) {
        bool all_above_150 = true;
        bool all_above_120 = true;

```

```

        for (int j = 0; j < 3; ++j) {
            if (hr[i + j] <= 150) all_above_150 = false;
            if (hr[i + j] <= 120) all_above_120 = false;
        }

        if (all_above_150) emergency = true;
        if (all_above_120) warning = true;
    }

    if (emergency) {
        cout << "Alert: EMERGENCY" << endl;
    } else if (warning) {
        cout << "Alert: WARNING" << endl;
    } else {
        cout << "Status: STABLE" << endl;
    }

    return 0;
}

```

Level 1 - (Advanced Logic)

1. Precision Irrigation Assistant

```

#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    double area, moisture;
    if (!(cin >> area >> moisture)) return 0;

    double volume = area * 5.0;

    if (moisture > 80.0) {
        volume = 0.0;
    } else if (moisture > 60.0) {
        volume *= 0.5; // 50% decrease
    } else if (moisture < 30.0) {
        volume *= 1.25; // 25% increase
    }
    // No change for 30 <= moisture <= 60

    cout << "Total: " << fixed << setprecision(1) << volume << "L" << endl;

    return 0;
}

```

2. Smart City Parking Fee

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int hours, isEV;
    if (!(cin >> hours >> isEV)) return 0;

    double fee = 0.0;

    if (hours <= 2) {
        fee = 30.0;
    } else {
        fee = 30.0 + (hours - 2) * 20.0;
    }

    // Surcharge
    if (hours > 12) {
        fee *= 1.10;
    }

    // EV Discount
    if (isEV == 1) {
        fee -= 15.0;
    }

    if (fee < 0) fee = 0; // Ensure no negative fees

    cout << "Final Fee: Rs. " << fixed << setprecision(2) << fee << endl;

    return 0;
}
```

3. Emergency Drone Deployment Optimizer

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace std;

struct Drone {
    int id, battery, x, y;
};

struct Request {
```

```

    int id, x, y, severity;
};

int get_distance(int x1, int y1, int x2, int y2) {
    return abs(x1 - x2) + abs(y1 - y2);
}

int main() {
    int d_count;
    cin >> d_count;
    vector<Drone> drones(d_count);
    for (int i = 0; i < d_count; ++i) {
        cin >> drones[i].id >> drones[i].battery >> drones[i].x >> drones[i].y;
    }

    int r_count;
    cin >> r_count;
    vector<Request> requests(r_count);
    for (int i = 0; i < r_count; ++i) {
        cin >> requests[i].id >> requests[i].x >> requests[i].y >>
requests[i].severity;
    }

    // Sort requests by severity (descending)
    // Note: The problem implies we process them in priority order
    sort(requests.begin(), requests.end(), [](const Request& a, const Request& b)
{
        if (a.severity != b.severity) return a.severity > b.severity;
        return a.id < b.id; // Secondary tie-break if needed (not explicitly
stated for same severity but good practice)
    });

    for (const auto& req : requests) {
        int best_drone_idx = -1;
        int min_dist = 1e9;

        for (int i = 0; i < d_count; ++i) {
            int dist = get_distance(drones[i].x, drones[i].y, req.x, req.y);
            int round_trip = 2 * dist;

            if (round_trip <= drones[i].battery) {
                if (dist < min_dist) {
                    min_dist = dist;
                    best_drone_idx = i;
                } else if (dist == min_dist) {
                    if (best_drone_idx == -1 || drones[i].id <
drones[best_drone_idx].id) {
                        best_drone_idx = i;
                    }
                }
            }
        }

        if (best_drone_idx != -1) {

```

```
        cout << "ASSIGNED: " << drones[best_drone_idx].id << " -> " << req.id  
<< endl;  
        drones[best_drone_idx].battery -= (2 * min_dist);  
        drones[best_drone_idx].x = req.x;  
        drones[best_drone_idx].y = req.y;  
    } else {  
        cout << "REQUEST_DROPPED: " << req.id << endl;  
    }  
}  
  
return 0;  
}
```